Custom Reinforcement Learning Agent Implementation

Environment: FrozenLake-v1 (Farama Gymnasium)

Algorithm: Tabular Q-Learning

Student: Samama Ali

Registration No.: FA23-BAI-024

University: COMSATS University Islamabad, Attock Campus

1. Problem Description

This project develops a custom reinforcement learning agent for the FrozenLake-v1 environment. The agent must navigate a frozen grid, avoid holes, and reach the goal. The objective is to learn a successful policy through interaction with the environment rather than using a manually programmed route.

2. Environment and State Space

FrozenLake-v1 is a discrete grid-world environment. In the standard 4x4 map there are 16 states and four actions: left, down, right, and up. The agent receives a positive reward for reaching the goal. Falling into a hole or ending unsuccessfully does not provide the goal reward. The environment can use slippery or non-slippery transitions depending on configuration.

3. Project Objectives

Implement a custom Q-learning agent; train the agent through repeated episodes; save learned Q-values; evaluate the learned policy; and visualize the agent's behavior using a GIF and training progress plot.

4. Methodology: Q-Learning

Q-learning is a model-free, off-policy reinforcement learning algorithm. It estimates the expected future return for every state-action pair using a Q-table.

Update rule:
Q(s,a) = Q(s,a) + alpha [r + gamma max Q(s',a') - Q(s,a)]

Here alpha is the learning rate, gamma is the discount factor, r is the immediate reward, s is the current state, a is the selected action, and s' is the next state.

5. Training Procedure

The agent initializes the Q-table, observes the current state, chooses an action using epsilon-greedy exploration, executes that action, receives a reward, and updates the Q-value. This process repeats until the episode ends. Over many episodes, the agent learns which actions are more useful for reaching the goal.

6. Hyperparameters

Parameter	Value found in project
Learning rate (alpha)	0.8
Discount factor (gamma)	0.95
Initial epsilon	1.0
Epsilon decay	0.9995
Minimum epsilon	0.05
Training episodes	100

Values above are extracted from the available Python source where simple numeric assignments are present. 'See source code' means the value could not be extracted automatically; it is not an assumed value.

7. Results and Evaluation

The trained agent's performance can be evaluated by running test episodes using the learned Q-table. The success rate is the percentage of evaluation episodes in which the agent reaches the goal.

Success rate = (successful evaluation episodes / total evaluation episodes) x 100

Saved Q-table shape: (16, 4). Non-zero values: 35.

8. Training Progress Visualization

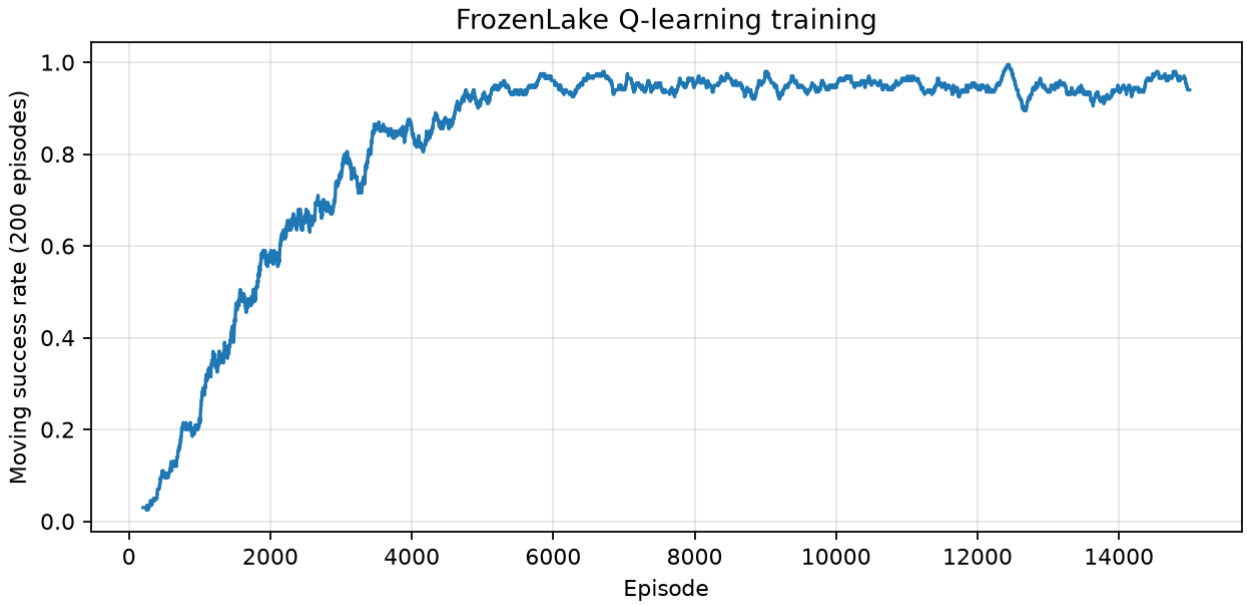


Figure 1: Training progress graph generated by the project.

9. Discussion

The training graph provides a visual representation of learning progress. Performance can vary with exploration rate, learning rate, discount factor, training duration, and whether the lake is slippery. A saved Q-table allows the learned policy to be reused without retraining.

10. Demonstration and Output Files

The project includes Python source code for the agent and training workflow, an executed Jupyter notebook, a saved Q-table, a training progress image, and an animated GIF demonstrating agent behavior.

File	Purpose
main.py	Training and evaluation workflow
agent.py	Custom Q-learning agent
utils.py	Helper and visualization functions
q_table.npy	Saved learned Q-values
training_progress.png	Learning progress graph
agent_demo.gif	Agent demonstration
FrozenLake_Training.ipynb	Jupyter notebook
requirements.txt	Required dependencies

11. Limitations and Future Improvements

Tabular Q-learning is suitable for small discrete environments but does not scale well to large or continuous state spaces. Future work could compare different hyperparameters, evaluate slippery and non-slippery environments, and implement Deep Q-Networks for more complex problems.

12. Conclusion

This assignment demonstrates the implementation of a custom reinforcement learning agent using Gymnasium and tabular Q-learning. The system supports training, policy storage, evaluation, and visual demonstration. It illustrates how an agent can learn from rewards and repeated interaction with an environment.

13. GitHub Repository

https://github.com/samamaali902-web/FrozenLake-Q-Learning-Assignment

14. References

Farama Foundation, Gymnasium Documentation; Watkins and Dayan, Q-learning (1992); Sutton and Barto, Reinforcement Learning: An Introduction (2018).